Le jeu des allumettes

Partie 1 : version console (brute)

Règle du jeu

Il y a **20 allumettes** sur la table. Le joueur et l'ordinateur jouent chacun leur tour, en commençant par le joueur. À chaque tour, on retire 1, 2 ou 3 allumettes. **Celui qui prend la dernière allumette a perdu.**

Travail demandé

Écris un programme Python qui :

- affiche le nombre d'allumettes restantes à chaque tour, à l'aide d'une chaîne de formatage ;
- demande au joueur combien il en prend, et redemande si son choix est interdit ;
- ne s'arrête pas si le joueur tape autre chose qu'un nombre (utilise `try` / `except`) ;
- fait jouer l'ordinateur avec la règle suivante : il prend `(allumettes - 1) % 4` allumettes, ou 1 si ce calcul donne 0 ;
- annonce le gagnant à la fin.

Question bonus

L'ordinateur suit une stratégie. Combien d'allumettes le joueur doit-il prendre au premier tour pour être sûr de gagner ?

Le jeu des allumettes

Partie 1 : version console (guidée)

Règle du jeu

Il y a **20 allumettes** sur la table. Le joueur et l'ordinateur jouent chacun leur tour, en commençant par le joueur. À chaque tour, on retire 1, 2 ou 3 allumettes. **Celui qui prend la dernière allumette a perdu.**

Étape 1 : les variables

Crée une variable `allumettes` qui vaut 20, et une variable `tour_joueur` qui vaut `True` (elle indique si c'est au joueur de jouer).

Étape 2 : la boucle de jeu

Le jeu continue tant qu'il reste des allumettes. Quelle boucle utiliser ? Avec quelle condition ? Au début de chaque tour, affiche le nombre d'allumettes restantes avec une chaîne de formatage, par exemple `f"Il reste {allumettes} allumettes"`.

Étape 3 : le tour du joueur

Si c'est au joueur de jouer, demande-lui un nombre avec `int(input(...))`. Ce nombre est interdit s'il est plus petit que 1, plus grand que 3, ou plus grand que le nombre d'allumettes restantes. Tant que la saisie n'est pas correcte, redemande.

Étape 4 : les erreurs de saisie

Si le joueur tape « trois » au lieu de 3, `int()` provoque une erreur `ValueError` et le programme plante. Place la saisie dans un bloc `try` et affiche un message d'erreur dans le bloc `except ValueError`. Comme dans le cours, utilise une variable `ok` qui passe à `True` quand la saisie est correcte.

Étape 5 : le tour de l'ordinateur

Sinon, c'est à l'ordinateur. Il prend `(allumettes - 1) % 4` allumettes. Si ce calcul donne 0, il en prend 1. Affiche combien il en prend.

Étape 6 : fin du tour

Retire les allumettes prises du tas, puis change de joueur. Astuce : `tour_joueur = not tour_joueur` inverse `True` et `False`.

Étape 7 : le gagnant

Après la boucle, le dernier à avoir joué a pris la dernière allumette, donc il a perdu. Mais attention, `tour_joueur` a déjà été inversé ! Si `tour_joueur` vaut `True`, qui a joué en dernier ? Affiche un message pour annoncer le résultat.

Squelette à compléter

Remplace chaque `...` par le bon code.

```
allumettes = 20
tour_joueur = True

while ... :
    ...
    if ... :
        ok = False
        while ... :
            try:
                prise = int(input("Combien en prends-tu (1 à 3) ? "))
                if ... :
                    ok = True
                else:
                    ...
            except ValueError:
                ...
    else:
        prise = ...
        if ... :
            prise = 1
        ...
    allumettes = ...
    tour_joueur = ...

if ... :
    ...
else:
    ...
```

Question bonus

L'ordinateur suit une stratégie. Combien d'allumettes le joueur doit-il prendre au premier tour pour être sûr de gagner ?

Le jeu des allumettes

Partie 2 : version graphique (brute)

Règle du jeu

Les règles sont les mêmes que dans la partie 1. L'ordinateur prend `(allumettes - 1) % 4` allumettes, ou 1 si ce calcul donne 0.

Travail demandé

Avec la bibliothèque `nsi_ui`, crée une interface graphique pour jouer au jeu des allumettes. Elle doit contenir :

- un texte qui affiche le nombre d'allumettes restantes ;
- trois boutons « Prendre 1 », « Prendre 2 » et « Prendre 3 », placés côte à côte ;
- un texte qui affiche les messages : ce que l'ordinateur a joué, un coup interdit, le gagnant.

Quand le joueur clique sur un bouton, le programme retire ses allumettes, puis l'ordinateur joue immédiatement. Une fois la partie terminée, les boutons ne doivent plus rien faire.

Le fichier `nsi_ui.py` doit se trouver dans le même dossier que ton programme.

Pour aller plus loin

- Ajoute un bouton « Rejouer » qui remet la partie à zéro.
- Utilise `set_background` pour colorer le message en vert quand le joueur gagne et en rouge quand il perd.

Le jeu des allumettes

Partie 2 : version graphique (guidée)

Règle du jeu

Les règles sont les mêmes que dans la partie 1. L'ordinateur prend `(allumettes - 1) % 4` allumettes, ou 1 si ce calcul donne 0.

Ce qui change

En version graphique, il n'y a plus de boucle `while` pour faire tourner le jeu : c'est la programmation **réactive**. Le programme attend que le joueur clique sur un bouton, et c'est ce clic qui déclenche la fonction de rappel. Le tour de l'ordinateur se fait juste après, dans la même fonction.

Étape 1 : les variables

Crée une variable `allumettes` qui vaut 20, et une variable `fini` qui vaut `False` (elle indique si la partie est terminée).

Étape 2 : l'interface

Dans un conteneur vertical, crée un `label` nommé `reste`, puis un conteneur horizontal avec les trois boutons, puis un `label` nommé `message`. Termine par `main_loop()`.

Étape 3 : un bouton avec un paramètre

La fonction `button` accepte un troisième paramètre : `button("Prendre 2", jouer, 2)` appelle `jouer(2)` quand on clique. Les trois boutons utilisent donc la même fonction `jouer(prise)`.

Étape 4 : modifier une variable dans une fonction

Pour qu'une fonction puisse modifier une variable créée en dehors d'elle, on écrit `global allumettes, fini` sur la première ligne de la fonction. Sans cette ligne, Python provoque une erreur.

Étape 5 : les cas impossibles

Dans `jouer`, vérifie d'abord si la partie est finie, puis si le joueur prend plus d'allumettes qu'il n'en reste. Dans ces deux cas, affiche un message avec `set_text(message, ...)` et ne fais rien d'autre.

Étape 6 : le coup du joueur, puis celui de l'ordinateur

Sinon, retire les allumettes du joueur. S'il n'en reste plus, il a perdu : affiche-le et passe `fini` à `True`. Sinon, l'ordinateur joue avec sa stratégie, puis on vérifie de la même façon s'il a pris la dernière.

Étape 7 : mettre à jour l'affichage

À la fin de `jouer`, mets à jour le label `reste` avec une chaîne de formatage.

Squelette à compléter

Remplace chaque `...` par le bon code. Le fichier `nsi_ui.py` doit se trouver dans le même dossier.

```
from nsi_ui import *

allumettes = 20
fini = False

def jouer(prise):
    global allumettes, fini
    if ... :
        set_text(message, ...)
    elif ... :
        set_text(message, ...)
    else:
        allumettes = ...
        if ... :
            ...
            fini = ...
        else:
            ordi = ...
            if ... :
                ordi = 1
            allumettes = ...
            if ... :
                ...
                ...
            else:
                set_text(message, f"...")
    set_text(reste, f"...")

begin_vertical()
reste = label("Il reste 20 allumettes")
begin_horizontal()
button("Prendre 1", jouer, 1)
...
...
end_horizontal()
message = label("À toi de jouer !")
end_vertical()
main_loop()
```

Pour aller plus loin

- Ajoute un bouton « Rejouer » qui appelle une fonction `rejouer()` : elle remet `allumettes` à 20, `fini` à `False` et met à jour les deux labels (pense à `global` !).
- Utilise `set_background(message, "green")` quand le joueur gagne et `"red"` quand il perd.